Lesson 13 Teacher's Key: A Hit That Reacts

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Using Godot 4's await keyword to pause a single function mid-execution without freezing anything else - a genuinely new language feature this course hasn't needed until now, introduced because this is the first moment a function needs to do something, then wait, then finish.

Level 1 - Full Worked Solution

- await get_tree().create_timer(0.1).timeout pauses ONLY this specific call of _on_hitbox_area_entered() - every other node, script, and function in the game keeps running normally.
- The explosion is instantiated and added to the scene BEFORE the await, so it starts playing out immediately and independently, with its own Timer managing its own cleanup.
- destroyed.emit() stays unchanged from Lesson 4/7 and runs BEFORE the await, so score/wave/win-check happen at exactly the same instant as last week. Only queue_free() moves to after the await - that is the one and only line actually delayed.
- hide() is removed this week - the Enemy needs to stay VISIBLE now so the new flash can actually be seen, a genuine, deliberate reversal of the hide-it-immediately-so-the-sound-can-finish technique Lesson 4's Level 3 established and Lesson 12 made mandatory.
- The old await $Hit.finished is replaced by await get_tree().create_timer(0.1).timeout - the function still waits for something before queue_free(), it's just waiting on the new flash timer instead of the sound now. $Hit.play() still runs immediately, so the hit sound still plays; it just isn't explicitly waited on anymore, so a hit sound longer than roughly 0.1 seconds could get cut off where it never could before.

Highlighted lines are new this week; everything else is code you already have.

```
# Enemy's hit-detection function (revised):
const FLASH_TEXTURE = preload("res://enemy_invader_white.png")

func _on_hitbox_area_entered(area):
    area.get_parent().queue_free()
    $Hitbox.set_deferred("monitoring", false)
    $Hit.play()
    var explosion = explosion_scene.instantiate()
    explosion.global_position = global_position
    get_tree().current_scene.add_child(explosion)
```

```
    texture = FLASH_TEXTURE
    destroyed.emit()
    await get_tree().create_timer(0.1).timeout
    queue_free()

# explosion.tscn's own script, attached to its Sprite2D root (NEW):
extends Sprite2D

func _ready():
    $Timer.timeout.connect(_on_timer_timeout)

func _on_timer_timeout():
    queue_free()
```



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

Why This Structure

- await is the right tool here specifically because the pause needs to happen INSIDE an existing function, without duplicating or restructuring the surrounding collision-detection code - a much smaller change than GameMaker or TIC-80 need for the same idea, since GDScript supports pausing execution directly.
- Giving the explosion its own scene with its own Timer (rather than having the Enemy manage the explosion's lifetime) keeps every object responsible for cleaning up only itself - the same discipline bullets and enemies have followed all course.

- The flash swaps texture rather than tinting modulate white, for the same reason Lesson 2's Recolor swaps sprites instead of tinting them: modulate multiplies against the sprite's existing color, and multiplying any color by white is a no-op - our enemy sprite is opaque red, so a white modulate tint would produce zero visible change.
- Replacing await $Hit.finished with await get_tree().create_timer(0.1).timeout (and dropping hide()) is a deliberate trade, not an oversight: last week's technique optimized for "let the sound finish, don't show anything broken-looking while it does" by hiding the Enemy immediately; this week's goal is the opposite - keep the Enemy visibly on screen, flashing, specifically so the flash can be seen. The two techniques can't both be satisfied by the same wait, so this lesson picks visibility over guaranteeing the sound's full length.

Level 2 / Level 3 Talking Points

- Ask students to explain, in their own words, why await here doesn't freeze the whole game the way a bug sometimes might - because it only pauses the specific function call it's written inside, not the entire engine.
- For students comparing engines: point out that Unity's Coroutine and Godot's await solve the identical problem (pause mid-function) with similar underlying mechanisms, while GameMaker and TIC-80 need a genuinely different structural approach (splitting into two events/checks) because their languages don't support pausing mid-script at all.
- Ask students why hide() had to come out this week even though it worked fine before - a concrete example of a technique that was exactly right for one problem (don't let the sound look broken) becoming exactly wrong for the next one (we now want the Enemy visible).

Level 2 - complete code:

```
# Level 2 is verification only - the Level 1 code already produces a
working flash and explosion, so no lines change.
# Checks: play a full round and confirm every single hit flashes and
explodes, with no enemy left stuck mid-flash and no explosion left on
screen;
# confirm score/enemies_remaining/wave-clear/win-check all still happen at
exactly the same instant they did before this lesson - only the enemy's
own final removal is delayed;
# confirm a second bullet hitting an already-flashing enemy does not
double-register a kill.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
# Enemy's hit-detection function - Level 3 flicker (replaces the single
texture swap):
const NORMAL_TEXTURE = preload("res://enemy_invader_red.png")

func _on_hitbox_area_entered(area):
    area.get_parent().queue_free()
    $Hitbox.set_deferred("monitoring", false)
    $Hit.play()
    var explosion = explosion_scene.instantiate()
    explosion.global_position = global_position
    get_tree().current_scene.add_child(explosion)
    texture = FLASH_TEXTURE
    await get_tree().create_timer(0.03).timeout
    texture = NORMAL_TEXTURE
```

```
    await get_tree().create_timer(0.03).timeout
    texture = FLASH_TEXTURE
    destroyed.emit()
    await get_tree().create_timer(0.04).timeout
    queue_free()

# explosion.tscn's own script - Level 3 expanding-scale animation (added
to its existing _ready()):
extends Sprite2D

func _ready():
    $Timer.timeout.connect(_on_timer_timeout)
    var tween = create_tween()
    tween.tween_property(self, "scale", Vector2(1.5, 1.5), 0.2)

func _on_timer_timeout():
    queue_free()
```

Common Student Mistakes

- Forgetting the await keyword and just calling get_tree().create_timer(0.1).timeout without it - this creates the timer but doesn't actually pause anything, so the function continues immediately.
- Instantiating the explosion AFTER the await instead of before, delaying the visual effect for no reason.
- Not guarding against a second bullet hitting the same enemy while it's mid-await, causing a double-destroyed.emit() call - freeing the bullet and disabling the Hitbox immediately (both before the await) is what prevents this.
- Calling area.queue_free() directly instead of area.get_parent().queue_free() - area IS the bullet's own Hitbox child node, not the Bullet scene's root. Destroying only the Hitbox leaves the actual Bullet (sprite, movement script, everything) still alive and on screen, just no longer able to detect collisions - an easy mistake to make since "area" sounds like it could mean the whole bullet.
- Checking area.is_in_group("bullets") before acting on a hit - this course's Godot bullets were never added to a group by that name at any point in Lessons 1-12, so a check like this silently always fails and the Enemy would never respond to being hit at all. Godot's dedicated-Hitbox design doesn't need a group check in the first place: the Enemy's own Hitbox Area2D only ever detects whatever its collision layers/masks let it detect, which in this project's setup is already just bullets.
- Explosions that appear but never disappear: two separate causes, both worth checking. The Timer's Autostart may be left off in the Inspector - Wait Time and One Shot don't start the countdown by themselves. Or explosion.tscn's own root node may be missing its own script entirely - enemy.gd only ever spawns the explosion and has no way to reach back into it later, so without a _ready() on the explosion itself connecting $Timer.timeout to queue_free(), nothing is ever listening for the timer to finish. Also confirm Timer is a DIRECT CHILD of whichever node the script is attached to (not nested under some other wrapper node) - $Timer only resolves a direct child, not a grandchild, which is an easy structural mistake if a student adds an extra Node2D above the Sprite2D instead of making the Sprite2D itself the scene's root.